

Pulpo User Guide

Installation system requirements and troubleshooting

Pulpo 1.0 repository edition | 29 September 2026

Pulpo is a Python governance and evidence kernel for agentic systems. It checks a proposed action against policy, issues narrowly scoped one-use permits, and keeps evidence for verification and reconciliation. This guide helps developers and technical operators install the core library, run a safe local example, and diagnose common problems.

Start here

Install Python and Git, create an isolated environment, install the local repository, and run the example on page 4. The example checks authorization only and does not modify a repository or call an external provider.

Section	Page
System requirements and scope	2
Install Pulpo	3
Run your first governance check	4
State and optional integrations	5
Troubleshoot installation and runtime	6
Maintain your installation and get help	7

How Pulpo works

Intent > Policy and authority > Decision > Permit > Execution > Evidence > Reconciliation

An intent describes who wants to do what, to which resource, and at what cost. An allowed decision can carry a permit bound to that exact intent. Consuming the permit is a one-use check; the application must separately perform the permitted work and reconcile its outcome. A successful executor response alone is not proof of the intended consequence.

Edition and boundaries

This guide covers **Ironnember/Pulpo1.0**, with package metadata version **0.1.0**. Pulpo Autonomous is a separate project. Its offline missions, physical-system adapters, and GPU benchmark instructions are not part of this installation.

Pulpo is currently a developer-facing project. Installing the library does not establish a production deployment, an independent human signer, or host isolation. Use a disposable local evaluation before connecting consequential services.

Source baseline: main commit d674dbb7a00b471d69bcc84d2f034cf74705e1a. Instructions are tied to this source snapshot; later versions may differ.

System requirements and scope

Component	Requirement or scope
Python	Python 3.11 or newer, with pip and virtual-environment support.
Core runtime	The base package declares no mandatory third-party runtime dependencies. SQLite support comes through Python.
Git	Required for the clone workflow and PulpoGit checks.
Operating system	Use a host with compatible Python and Git. Commands are supplied for Windows PowerShell and macOS/Linux shells; this is not an OS certification matrix.
Storage and permissions	A writable environment and project directory. Persistent deployments also need a protected, writable database location.
Network	Needed to obtain the repository and install packages. The local core example does not need external services.
GPU	Not required for the core library or example. No GPU extra is declared in this source baseline.

Hardware planning

The baseline does not publish a minimum CPU, RAM, or free-disk specification. Size the environment for your own audit volume, retention, and optional services. Begin with the core example on a development computer and measure before setting operational capacity targets.

Optional dependency profiles

Authority: the `authority` extra pins `cryptography==50.0.1` for asymmetric verification. Installing it does not create a human approval service or provision trusted signing keys.

MCP: the `mcp` extra requests `mcp[cli]>=2, <3`. It supports the optional transport layer; it is not required for core governance checks.

Separate services: `authority-service` and `custody-service` have their own setup and trust requirements. Follow their repository documentation before deploying them. Docker, cloud credentials, and provider accounts are not prerequisites for the basic example.

Install Pulpo

Open a terminal in a directory where you keep development projects. Use the block for your operating system. These commands install the source checkout, avoiding confusion with similarly named packages.

Windows PowerShell

```
py -3 --version
git --version
git clone https://github.com/Ironnember/Pulpo1.0.git
cd Pulpo1.0
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -e .
```

Confirm that the first command reports Python 3.11 or newer. The explicit environment path works without activating a PowerShell script. For the remaining commands in this guide, replace `python` with `.\.venv\Scripts\python.exe`.

macOS or Linux

```
python3 --version
git --version
git clone https://github.com/Ironnember/Pulpo1.0.git
cd Pulpo1.0
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
```

Confirm Python is 3.11 or newer. On Linux, if virtual-environment creation reports that `venv` or `ensurepip` is missing, install the matching Python `venv` package using your operating system package manager, then retry.

Confirm the installation

```
python -c "import sys, pulpo; print(sys.version); print(pulpo.__file__)"
python -m pip show pulpo
python -m pip check
git rev-parse HEAD
```

The import path should point to this checkout, and pip should report the installed package. Record the commit with your test results. To reproduce this guide exactly, use a separate clean checkout of the source-baseline commit listed on page 1, then reinstall there.

Keep the repository directory: editable installation reads its source files directly. Do not move or delete it while using that environment. When returning later on macOS/Linux, reactivate the environment; on Windows, keep using the explicit Python path.

Run your first governance check

In the repository directory, save the following as `quickstart.py`. This is an in-memory demonstration, not a file-writing adapter.

```
from pulpo import GovernanceKernel, Intent, Policy

kernel = GovernanceKernel(
    Policy(
        allowed_actions=frozenset({"read", "write"}),
        max_cost=100,
    )
)
intent = Intent("agent:builder", "write", "repo:README.md", cost=5)
decision = kernel.evaluate(intent)
print("Decision:", decision.outcome)

if decision.outcome == "allow":
    print("First use:", kernel.consume(decision.permit, intent))
    print("Replay:", kernel.consume(decision.permit, intent))

print("Audit valid:", kernel.verify_audit())

python quickstart.py
```

Expected output:

```
Decision: allow
First use: True
Replay: False
Audit valid: True
```

Interpret the result

Allow means this policy accepted the proposed intent. **First use True** means the permit was consumed for the exact intent. **Replay False** confirms it cannot be consumed again. **Audit valid True** confirms the local audit chain passes the kernel check.

No README file is written by this code. Your application remains responsible for the separately permitted execution and its evidence. For high-impact work, use the configured approval path; do not replace verification with a caller-supplied approval flag.

Run repository checks

```
python -m unittest discover -s tests -v
python -m unittest discover -s tests -p test_persistence.py -v
```

Run these from the repository root. Review failures and skips, not just the final summary. Optional dependencies and platform-specific proof requirements may affect results. A passing local test suite does not certify an external deployment.

State and optional integrations

Choose the state lifetime

The default in-memory state is suitable for short demonstrations. To retain kernel replay and audit state across process restarts, use `SQLiteKernelState` with a stable database path:

```
from pulpo import GovernanceKernel, Policy, SQLiteKernelState
```

```
state = SQLiteKernelState("kernel.sqlite3")
try:
    kernel = GovernanceKernel(
        Policy(allowed_actions=frozenset({"read"}), max_cost=10),
        state=state,
    )
    print("Audit valid:", kernel.verify_audit())
finally:
    state.close()
```

This initializes or opens a local store; it is not a complete approval-backed restart configuration. Production integration must preserve the appropriate policy, verifier trust, and permit-secret configuration. Do not copy test signing secrets into a deployment.

Protect the database from untrusted writers. Use a consistent SQLite backup procedure; copying only the main file during active writes can miss journal state. A database that passes its chain check is not automatically protected against rollback to an older valid snapshot.

Install an optional profile only when needed

```
python -m pip install -e ".[authority]"
python -m pip install -e ".[mcp]"
```

You may install either extra independently. If dependency resolution fails, inspect the exact Python version and package-index availability; do not silently substitute an incompatible version.

MCP reader setup

The repository plugin launches a stdio reader over a frozen snapshot. A trusted Pulpo process must export the snapshot through the existing exporter. The operator then supplies its absolute path; the reader does not create canonical state or issue permits.

```
python -m pulpo.mcp_plugin --snapshot ABSOLUTE_SNAPSHOT_PATH
```

Replace the placeholder with a real absolute file path, quoted if it contains spaces. The reader checks the schema, size, link restrictions, and private-file rules. On Windows, ensure the MCP client launches the environment Python rather than assuming `python3` resolves correctly.

The two tools construct intent proposals and return frozen evidence metadata. A snapshot can be stale. Local plugin startup does not prove public hosting or production client connectivity. See the linked MCP references on page 7.

Troubleshoot installation and runtime

Python or Git is not recognized

Install the missing program, reopen the terminal, and repeat the version checks. On Windows try `py -3 --version`; on macOS/Linux try `python3 --version`. Confirm Python is at least 3.11.

No module named pulpo or the wrong package loads

Use the same environment Python for installation and execution. From the repository root, rerun `python -m pip install -e .` and print `pulpo.__file__`. If it points to another checkout, correct the environment before continuing.

PowerShell blocks environment activation

Use `.\.venv\Scripts\python.exe` directly as shown on page 3. Changing system execution policy is unnecessary for this workflow.

Optional dependency will not install

Capture the full pip error. Check the selected interpreter, package index, network access, and availability of the version declared in `pyproject.toml`. Core examples do not require the optional authority or MCP extras; diagnose those profiles separately.

An action is denied or a permit will not consume

Inspect `decision.outcome` and `decision.reason`. Check the action, resource, cost, agent grant, and approval requirements. Consume only the permit for the unchanged intent. A used permit must fail on replay; changing the resource or other bound fields does not reuse the authorization.

StateIntegrityError or audit verification failure

Stop consequential work and preserve the database and error details. Check whether the wrong file was opened, external software modified it, or recovery left inconsistent state. Do not delete evidence or rewrite hashes to force startup. Investigate and recover through a trusted, documented process.

Database is locked or the state directory is unwritable

Check directory permissions, disk space, and competing writers. Close unused connections with `state.close()`. Prefer a local protected database location over an actively synchronized folder for evaluation. Do not delete lock or journal files while a process is using the database.

MCP snapshot rejected or server not starting

Check the absolute path, exact exported schema, permissions, file size, and symbolic-link restrictions. Re-export from the trusted source instead of hand-editing evidence. Verify the MCP extra is installed in the interpreter launched by the client. The stdio process expects an MCP client, not a web browser.

GPU instructions or Autonomous modules do not work

Confirm the repository identity. This baseline has no GPU extra and does not contain the Autonomous core/domain modules. Instructions for another branch or project cannot be assumed to apply. A path such as `/workspace` exists only where that directory was created, often inside a container.

Maintain your installation and get help

Update in a controlled environment

Record the current commit and preserve your changes and state before updating. Review the new source and dependency requirements, install the selected revision in a separate environment, and rerun the example and relevant tests. Do not point an untested revision at your only copy of a live database.

Inspect local source state

```
git status --short
git rev-parse HEAD
python proofs/git_clarity/run.py
```

PulpoGit reports local checkout state. It does not fetch remote changes, run tests, or grant authority. Remote-tracking references can be stale. An exported report must be written outside the repository when using its output option.

Collect a useful support report

Include the operating system, Python version, repository URL and commit, optional profiles installed, exact command, expected result, actual result, and complete traceback. Add a minimal synthetic reproduction and relevant test results. Remove credentials, signing material, sensitive intent data, and private database contents before sharing.

Report reproducible project problems through the repository [issue tracker](#). Do not publish private signing credentials or sensitive evidence in an issue.

Source references

The following links target the source snapshot used for this guide. Requirements are recorded from package metadata; workflows are derived from source and examples. Operational sizing guidance is advisory, not a published hardware minimum.

[Project overview](#) | [README.md](#)

[Python version and optional dependencies](#) | [pyproject.toml](#)

[Kernel behavior](#) | [pulpo/kernel.py](#)

[Persistence and restart tests](#) | [tests/test_persistence.py](#)

[SQLite persistence boundary](#) | [docs/PERSISTENCE.md](#)

[MCP plugin and snapshot handoff](#) | [docs/PULPO_AGENT_PLUGIN.md](#)

[MCP capability boundary](#) | [docs/MCP_BOUNDARY.md](#)

[Authority configuration](#) | [docs/AUTHORITY.md](#)

[Local Git clarity](#) | [proofs/git_clarity/README.md](#)

Documentation can lag executable code. Resolve conflicts against the exact revision and its tests before making operational claims.